EduGenie

Project Description:

EduGenie is a lightweight AI-powered educational assistant designed to simplify and enhance the learning experience through the power of Generative AI. Built for students, self-learners, and educators, the platform analyzes learner questions and study materials to provide accurate answers, simplified concept explanations, topic-specific quizzes, concise summaries, and personalized learning path recommendations.

Built using **FastAPI**, a responsive **HTML/CSS** frontend, **Natural Language Processing (NLP)**, and the **Google Gemini API**, the application enables learners to understand concepts, assess their knowledge, and follow structured learning roadmaps with confidence. The system also maintains learner interaction history, collects feedback, and is optimized to run efficiently on resource-constrained devices such as the Mac M1.

Scenarios

Scenario 1:

A student wants to learn about oceans and rivers. They ask, “Which is the largest ocean?” EduGenie instantly provides an accurate answer along with additional educational context, helping the student understand the topic beyond a simple fact.

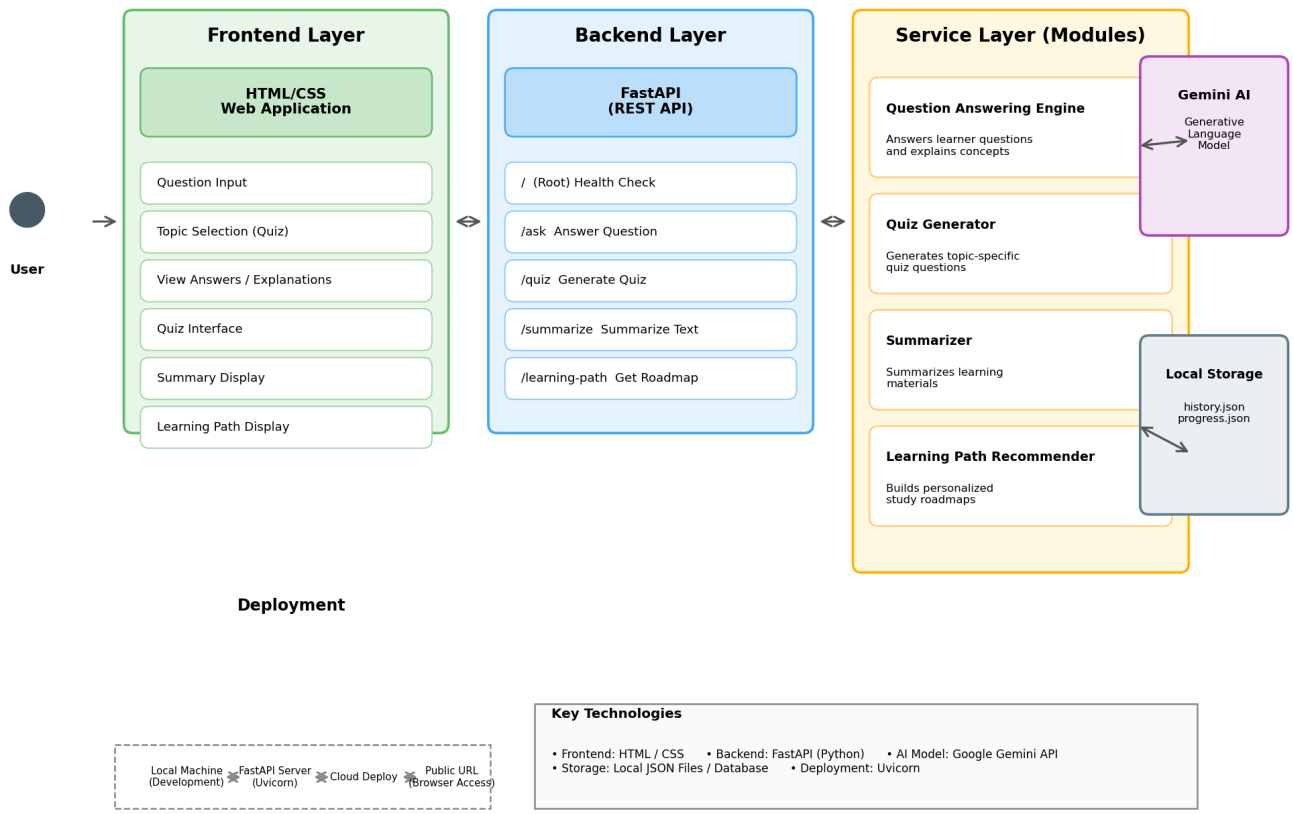
Scenario 2:

A student studying the Pythagoras Theorem wants to assess their understanding. They select “Generate Quiz,” and EduGenie creates topic-specific questions for self-evaluation, helping the student identify gaps in their knowledge.

Scenario 3:

A learner interested in SQL requests a structured learning roadmap. EduGenie generates a personalized learning path covering beginner, intermediate, and advanced topics with study recommendations and progression guidance.

Technical Architecture



Description:

EduGenie follows a modular three-tier architecture.

- The **Frontend Layer** consists of a responsive HTML/CSS-based user interface.
- The **Backend Layer** is implemented using FastAPI and handles request processing and API routing.
- The **Service Layer** contains reusable modules for question answering, quiz generation, summarization, and learning path recommendation.

The architecture enables scalability, maintainability, and modular development.

Pre-requisites

- Python Programming Knowledge
- FastAPI Framework Knowledge
- HTML/CSS Fundamentals
- Natural Language Processing Concepts
- Prompt Engineering
- Google Gemini API / Generative AI Concepts
- REST API Concepts
- Git and GitHub Basics
- Virtual Environment Management

Project Workflow

Milestone 1: Environment Setup and Architecture Design

Activity 1.1: Define Project Requirements

- Identify learner requirements.
- Define educational use cases.
- Finalize project scope.

Activity 1.2: Design System Architecture

- Create architectural diagrams.
- Define interactions between frontend, backend, and services.

Activity 1.3: Set Up Development Environment

- Install Python.
- Create virtual environment.
- Install dependencies.

Activity 1.4: Create Project Structure

Create folders for:

- app/
- services/
- models/
- routers/
- frontend/
- tests/

Milestone 2: Backend Development

Activity 2.1: Implement FastAPI Backend

Create API endpoints:

- /
- /ask
- /quiz
- /summarize
- /learning-path

Activity 2.2: Develop Question Answering Module

Implement AI-based question answering using the Google Gemini API.

Functions:

```
answer_question()
```

Responsibilities:

- Analyze learner questions.
- Generate accurate, contextual answers.

Activity 2.3: Develop Quiz Generation Module

Functions:

```
generate_quiz()
```

Responsibilities:

- Generate topic-specific quiz questions.

- Match question difficulty with learner level.

Activity 2.4: Develop Summarization Module

Functions:

```
summarize_text()
```

Responsibilities:

- Condense learning materials into concise summaries.
- Preserve key concepts and terminology.

Activity 2.5: Develop Learning Path Recommendation Module

Functions:

```
generate_learning_path()
```

Responsibilities:

- Build beginner-to-advanced roadmaps.
- Provide study recommendations and progression guidance.

Milestone 3: Frontend Development

Activity 3.1: Develop HTML/CSS Interface

Create user-friendly pages for:

- Question input.
- Quiz topic selection.
- Answer and explanation display.
- Summary display.
- Learning path display.

Activity 3.2: Add User Interaction Features

Implement:

- Interactive quiz interface.
- Conversation/learning history.
- Loading animations.
- CSS animations for a smoother experience.

Milestone 4: Data Storage and Logging

Activity 4.1: Learning History

Store generated interactions in:

```
history.json
```

Responsibilities:

- Save generated answers, quizzes, and summaries.
- Retrieve historical learning sessions.

Activity 4.2: Feedback Collection

Store learner feedback in:

```
feedback.json
```

Responsibilities:

- Save learner ratings on answers and quizzes.
- Analyze learner preferences to improve recommendations.

Milestone 5: Testing and Validation

Activity 5.1: Unit Testing

Create test files:

```
test_qa_engine.py
test_quiz_generator.py
test_routes.py
```

Use:

```
pytest -v
```

Validate:

- API functionality.
- NLP and AI modules.
- Quiz generation accuracy.
- Route responses.

Activity 5.2: Swagger API Testing

Access Swagger UI:

```
http://127.0.0.1:8000/docs
```

Test:

- Ask endpoint.
- Quiz endpoint.
- Summarize endpoint.
- Learning-path endpoint.

Milestone 6: Deployment and Documentation

Activity 6.1: Local Deployment

Run FastAPI:

```
python -m uvicorn app.main:app --reload
```

Open the frontend:

```
frontend/index.html
```

Activity 6.2: Version Control

Upload project to GitHub.

Repository includes:

- Source code.
- Documentation.
- Images.
- Tests.
- README.

Exploring the Application Pages

Home Page

Description:

The homepage allows learners to:

- Ask a question.
- Select a topic for quiz generation.
- Paste text to summarize.
- Request a personalized learning path.

Answer Section

Description:

Displays the AI-generated answer along with additional educational context.

Example:

```
“The Pacific Ocean is the largest and deepest ocean, covering about a third of the Earth's surface.”
```

Quiz Section

Description:

Displays AI-generated, topic-specific quiz questions for self-evaluation.

Example:

```
“What is the relationship between the sides of a right-angled triangle according to the Pythagoras Theorem?”
```

Learning Path Section

Description:

Displays a structured roadmap covering beginner, intermediate, and advanced topics.

Example:

```
SQL Basics → Joins & Subqueries → Indexing & Optimization → Advanced Query Design
```

History Section

Description:

Displays previously generated answers, quizzes, summaries, and learning paths.

Conclusion

EduGenie successfully demonstrates how Artificial Intelligence and Natural Language Processing can improve the learning experience. The system assists learners by answering questions, simplifying concepts, generating quizzes, summarizing materials, and providing personalized learning paths.

The modular architecture built using FastAPI and a responsive HTML/CSS frontend ensures maintainability and scalability. Future enhancements include cloud deployment, user authentication, database integration, multilingual support, and an analytics dashboard for tracking learner progress.

This project highlights the practical application of Generative AI in making learning more interactive, accessible, and efficient for students, self-learners, and educators alike.